

Features — current state

Everything the Customer Portal can do right now. Items marked **(placeholder)** are wired up and reachable but don't have real functionality yet.

1. Authentication

Login only — no public registration.

Feature	Status	Route	Notes
Email + password login	☐ Active	GET/POST /login	Backed by Laravel Fortify
"Remember me"	☐ Active	—	Checkbox on the login form
Logout	☐ Active	POST /logout	From the user menu (top-right)
Registration	☐ Removed	—	Disabled in Fortify; no sign-up anywhere
Forgot / reset password	☐ Active	/forgot-password , /reset-password	"Forgot password?" link on login
Email verification	☐ Active	/email/verify	Dashboard & Chat require a verified email
Two-factor authentication (2FA)	☐ Active	under Settings → Security	TOTP + recovery codes
Passkeys (WebAuthn)	☐ Active	under Settings → Security	Passwordless login option

Rate limiting: The login / 2FA / passkey throttles were **removed** — you can attempt login as many times as you want. (The password-change endpoint under settings still has a 6/min throttle, which we left in place.) Separately, the **app endpoints** are throttled per-user to stop runaway loops and abuse — chat send, chat search, and the integration endpoints — with generous, .env -tunable limits (RATE_LIMIT_* , config/ratelimits.php) that normal use never reaches.

2. Login screen (custom design)

A bespoke, modern "AI product" login:

- Full-bleed **deep-ocean dark** scene with an **animated sonar-ping background** (HTML Canvas; respects prefers-reduced-motion).
- **Frosted-glass** sign-in card with a cyan accent and a glowing Sign-in button.
- Monospace "instrument" labels (e.g. CW GLOBAL PEOPLE · CUSTOMER PORTAL).

- Component: `resources/js/components/SonarBackground.vue` , layout: `resources/js/layouts/AuthLayout.vue` , page: `resources/js/pages/auth/Login.vue` .

3. Navigation

- **Top header bar** (not a sidebar). Layout: `AppHeaderLayout.vue` .
- Brand: **CW Global People** (logo text in `AppLogo.vue`).
- Nav items: **Dashboard, Chat, Projects, Integrations**.
- Right side: a **chat search** (icon or ⌘/Ctrl-K, see §7) and a user menu (settings, logout).
- The header is **full-width** (logo flush-left, gold active-tab underline). All main pages now use the full window width for a consistent, maximized layout.
- The old **Repository** and **Documentation** links were removed.

4. Pages

Page	Route	Status
Dashboard	<code>/dashboard</code>	☐ Token-usage meter + conversation/project/skill counts
Chat	<code>/chat</code>	☐ Working — AI chat powered by the Claude API (see §7)
Integrations	<code>/integrations</code>	☐ MCP servers + n8n live ; other cards placeholder
Settings → Profile	<code>/settings/profile</code>	☐ Update name & email, delete account
Settings → Security	<code>/settings/security</code>	☐ Change password, manage 2FA & passkeys
Settings → Appearance	<code>/settings/appearance</code>	☐ Light / dark / system theme
Settings → Skills	<code>/settings/skills</code>	☐ Create / import / manage reusable instruction presets

`/` (root) redirects: → `/dashboard` when logged in, → `/login` otherwise.

5. Account management (Settings)

- **Profile**: change name and email; delete account (requires verified email).
- **Security**: change password; enable/disable two-factor auth (with QR code and recovery codes); register and manage passkeys. Viewing this page requires re-confirming your password.
- **Appearance**: switch between light, dark, and system themes (persisted).
- **Skills**: create reusable **instruction presets**, add ready-made ones from a **starter library** (config `config/skills.php`), or **import a SKILL.md** (front-matter `name / description` + body). Each skill = name, emoji icon, description, and instructions. Per-user and owner-checked. Pick one above the chat composer to apply it (see §7). Route group `settings/skills` (`SkillController`).

7. Chat — AiMe BOT (Claude AI)

A working AI chat assistant called **AiMe BOT** on the `/chat` page, powered by the **Claude API**.

- **Modern UI:** branded gradient bot avatar, "Online" status, per-message avatars (bot + user initials), a typing indicator, and a soft ambient cyan→indigo glow behind the empty state (so it isn't flat in dark mode).
- **Personalized empty state:** a time-of-day greeting using the logged-in user's name — "Good morning/afternoon/evening, {name}" — with a simple "Hi, I'm AiMe BOT. Ask me anything." subtitle.
- **Model switching is live:** changing the model in the header applies to the next message (no refresh); you can even switch models mid-conversation.

Note: the chat follows the app's light/dark theme (Settings → Appearance). The dark background is dark mode, not the chat itself.

- **Model:** picked by the user from a dropdown in the chat header. Ships with **8 Claude models** — Opus 4.8 / 4.7 / 4.1, Sonnet 5 / 4.6 / 4.5, Haiku 4.5, and Fable 5. Defaults to `ANTHROPIC_MODEL`; the choice is remembered in the browser (`localStorage`). The allowlist lives in `config/services.php` (`anthropic.models`) and is validated server-side — add/remove freely.
- **Skills:** a **skill selector** above the composer applies one of your saved Skills (Settings → Skills, §5) to the chat — its instructions are injected into the system prompt (single skill at a time). The choice is stored on the conversation (`conversations.skill_id`) and restored when you reopen it. Manage skills in Settings.
- **Search your chats:** the header **search icon** (or ⌘/Ctrl-K) opens a dialog that searches your conversations by **title or message content** and shows a matching snippet; clicking a result opens that chat (project chats open in their project). Backed by `GET /chat/search`, scoped to the current user.
- **How it works:** the reusable `ChatPanel.vue` sends the new message + conversation id to `POST /chat/message`. The server-side `ChatController` loads history from the DB, calls Claude via the official `anthropic-ai/sdk`, persists both sides, and returns the reply as JSON. **The API key never reaches the browser.**
- **Auth:** the endpoint is behind `auth` + `verified` middleware.
- **Config (all in `.env`):** `ANTHROPIC_API_KEY` (required), `ANTHROPIC_MODEL` (default model), `ANTHROPIC_MAX_TOKENS` (reply length), and `ANTHROPIC_SYSTEM_PROMPT` (the guardrails/persona — single-line override; the multi-line default lives in `config/services.php`). The model **allowlist** is also in `config/services.php` (`anthropic.models`). Without a key the chat shows a friendly "not configured yet" message instead of erroring.
- **Layout:** the standalone `/chat` page is **full-bleed** — the **conversation sidebar** (new chat, history list, delete) sits flush at the far left and the chat fills the full window width (no centered gutters). The conversation and composer now span the **full panel width** (no narrow reading column), so there's no dead space left/right. Responsive: the sidebar becomes a drawer on mobile. (Inside a Project workspace the same `ChatPanel` renders as a bordered card beside the knowledge panel.)
- **Saved chats (persistence):** conversations and messages are stored in the database **per user** (`conversations` + `messages` tables). Chats survive a refresh, appear in the sidebar (most-recent first), and reopen with their full history. The server is the source of truth — each turn sends only the new message + conversation id, and history is loaded from the DB. All endpoints are scoped to the authenticated user.
- **File uploads (images + PDFs):** attach files with the composer paperclip; Claude reads them natively. Files are **re-sent every turn** (stored on the message), so follow-up questions keep the document in view. Configurable in `.env` (`ANTHROPIC_UPLOADS_*`): enable/disable, max files, max size, and allowed extensions. Word/Excel are **not** supported yet (would need text extraction) — see `roadmap.md`.

- **Token usage:** each reply's input/output tokens (from the Claude API `usage`) accumulate on the conversation and show as a small **"N tokens"** pill in the composer footer (hover for the in/out breakdown). Resets on New chat.
- **Per-user token budget:** every user has a rolling allowance (default **1,000,000 tokens / 30 days**, configurable via `USAGE_*`). Usage is charged per reply; once spent, sending is blocked with a "resets on " message until the window rolls over. The **Dashboard** shows the live meter. (`TokenBudget` .)
- **Prompt caching + history trimming:** the system prompt is cached (`cache_control`) so repeat turns are cheaper, and only the most recent `ANTHROPIC_HISTORY_LIMIT` messages (default 40) are replayed each turn, keeping long conversations' context and cost bounded.
- **Streaming replies:** the assistant's reply **streams in token-by-token** over Server-Sent Events (`POST /chat/stream`) — a "Thinking..." indicator shows only until the first token, then the text fills in live. The message, token totals, and n8n event are all persisted server-side once the stream finishes (identical bookkeeping to the non-streaming path, which remains at `POST /chat/message`). File uploads stream too.

6. Projects (Claude-style workspaces)

Named workspaces that give the assistant lasting context for a specific job — modeled on Claude.ai Projects.

- **Projects list** (`/projects`) — create projects, open them, and **delete** them (hover a card → trash → click again to confirm).
- **Project workspace** (`/projects/{id}`) — the chat experience scoped to the project (its own conversation history) with a **visible right-hand panel** (`ProjectKnowledge.vue`) to edit the project's **Name** and **Instructions**, plus a **Delete project** button. On small screens the panel opens as a drawer.
- **Instructions are injected** into the system prompt for every chat in the project (appended to the base guardrails), so the assistant behaves consistently. *(A per-project **Memory** field was removed — not needed for now. The DB column is retained but unused.)*
- **Scoping:** a conversation optionally belongs to a project (`conversations. project_id`). Project chats appear only in that project; standalone `/chat` shows only non-project chats. All endpoints are owner-checked.
- Backend: `ProjectController` , `Project` model, `projects` table. UI: `projects/Index.vue` , `projects/Show.vue` .
- **Not yet:** project-level file attachments (per-message uploads exist in chat, §7) — see `roadmap.md`.

8. Integrations

Connect AiMe BOT to outside tools. The page (`/integrations`) groups connectors by category (Communication, CRM, Files & documents, Automation, ERP & business systems, Productivity & data). Each card has a **Setup guide** modal with numbered steps.

- **MCP servers (live — native tools).** Connect a **Model Context Protocol** server (Slack, GitHub, Notion, ... anything exposing MCP) with a name, URL, and optional bearer token; the assistant can then **call that server's tools** during chat. Anthropic runs the tool calls **server-side** via the MCP connector. Per-user, token **encrypted at rest**, URL **SSRF-guarded**. Enable/disable/delete each server. **Tool turns stream** like normal chat, and the composer shows **"Using <server>..."** while a tool runs. (MCP turns send text-only history — per-message image/PDF passthrough is still chat-only.) Backend: `McpServerController` , `McpServer` model; streamed via `beta.messages` in `ChatController::stream` . Config: `ANTHROPIC_MCP_BETA` .

- **n8n (live).** Paste the **Production URL** of an n8n **Webhook** node (plus an optional shared secret, sent as a header). AiMe BOT **POSTs** a `chat.completed` **event** to that URL after every reply, so your n8n workflow can react. Use **Send test** to fire a `test.ping` and see the HTTP status, and **Disconnect** to remove it. Connections are **per-user** and stored **encrypted** (`user_integrations` table).
- **Everything else** (Slack, Email, GHL, HubSpot, Salesforce, Google Drive/Sheets, Webhooks, Zapier, NetSuite, Calendar, Database, Code repos) is a **placeholder** with a working setup guide but a disabled Connect button.
- Backend: `IntegrationController` , `N8nDispatcher` , `UserIntegration` model.
- **Config (all in `.env`):** `INTEGRATIONS_LIVE` (comma-separated provider keys whose Connect works), `INTEGRATION_N8N_TIMEOUT` , `INTEGRATION_N8N_SECRET_HEADER` — defaults in `config/integrations.php` .

7. Theming

- **Brand palette: navy + gold** (aligned with [cwglobalpeople.com](https://www.cwglobalpeople.com)). Light mode is white with a navy primary and gold accents; dark mode uses navy-tinted surfaces with gold primary CTAs. Brand tokens live in `resources/css/app.css` (`--brand-gold` , `--brand-navy` , ...) and are used across avatars, gradients, and the login screen.
- The product is labelled **CWGP-AiMe** on the login screen (the header brand name remains **CW Global People**).
- Light / dark / system mode, with the preference remembered across sessions.
- Built on Tailwind CSS v4 + shadcn-vue design tokens.

What's NOT built yet

- **Office document uploads** (Word/Excel/CSV) — chat now accepts **images + PDFs** natively (\$7), but Office formats would need server-side text extraction. Also **project-level files** (a persistent per-project knowledge base, vs. per-message attachments) and **auto-updating memory** — see [roadmap.md](#).
- **Streaming** chat responses (replies arrive all at once after a wait).
- Real **Dashboard** content — only placeholder cards.
- A custom brand **logo icon** — still the default Laravel mark; only the text was rebranded.